

Pangea Toolkit — Tool Structure

Ninaad Adhvaryu

May 2026

Contents

Pangea Toolkit	1
1. Overview	1
2. Architecture	2
3. Instructions — the v6 system prompt	3
4. Context the tool has access to	5
5. Iteration discipline	6
6. Sample input and output	6
7. nhsjs-tools — the submission-formatter component	8
Scope and authorship	9

Pangea Toolkit

A versioned, corpus-grounded research-ideation system for Indian high-school students working toward publishable scientific projects. Deployed as a per-student Claude Project. Built and maintained solo at Athena Education.

Repositories

- This toolkit: github.com/adhvaryu-athena/pangea-toolkit
- Submission-formatter component: github.com/adhvaryu-athena/nhsjs-tools

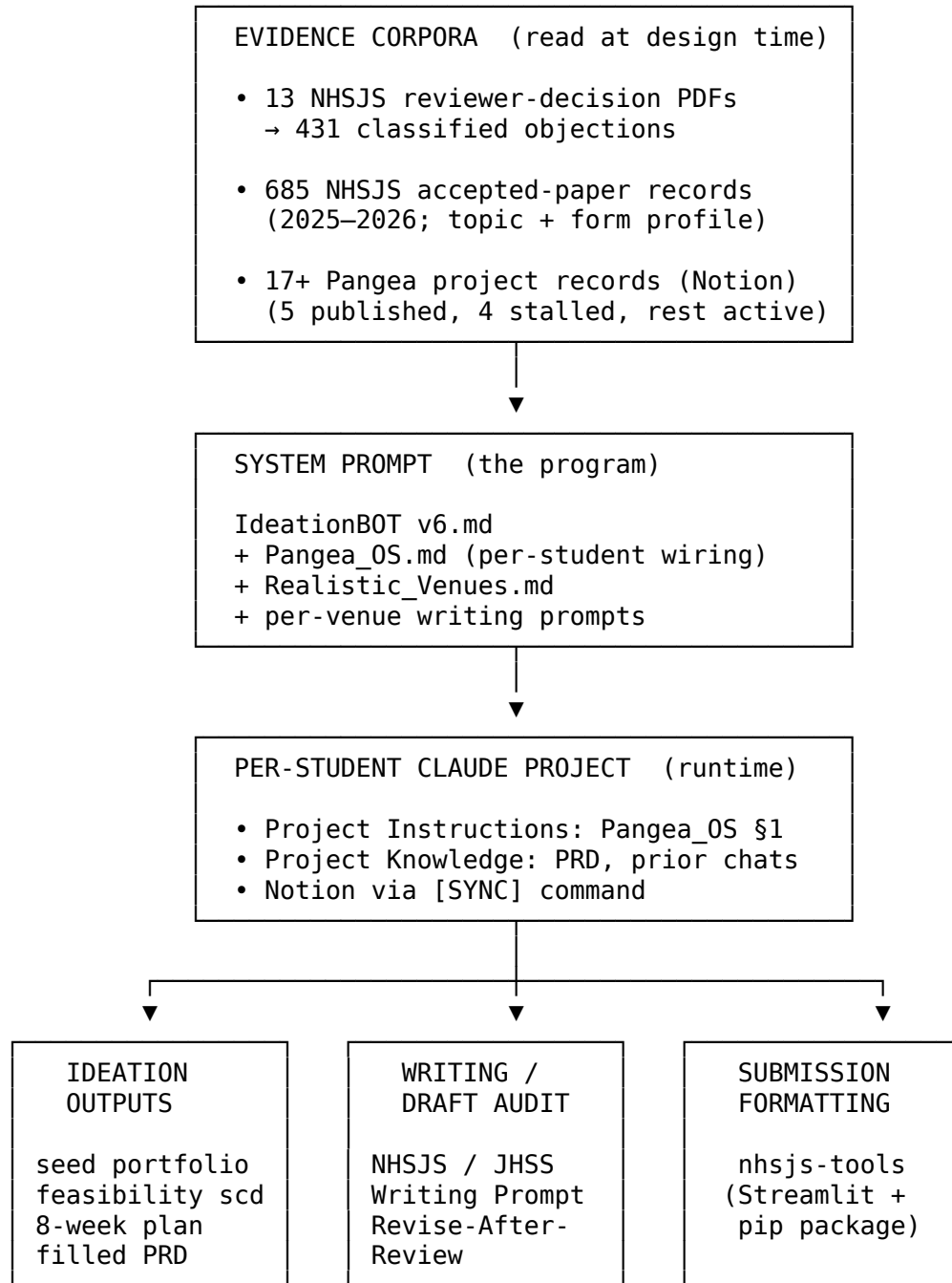
1. Overview

The Pangea Toolkit is a set of system prompts and reference files that takes a student from a vague research interest (“I want to do something with neural networks”) to a venue-calibrated, novelty-screened research question and a full Project Requirements Document. It is not a codebase. The core artifact is the IdeationBOT system prompt — currently at v6, with v3, v4, v5, v5.1, v5.2 preserved in `_archive/` — pasted into a fresh Claude chat at the top of an ideation session. Around it sit per-venue writing tools, a reviewer-pattern intelligence layer, a canonical venue reference (`Realistic_Venues.md`), and a `Pangea_05.md` operations file that wires it all into a per-student Claude Project.

I built it because mentor-led ideation at Athena’s Pangea program was running on intuition. A mentor would propose ideas, the student would pick one, three weeks later the data wouldn’t operationalize or the form was structurally indistinguishable from work the target journal had already published. The toolkit replaces that with a single design-time step that screens form-saturation, names the creative move, locks the primary outcome, and pre-commits a fallback framing before any code gets written.

The toolkit sits between two human steps in the mentor workflow: after the mentor has talked to the student about interests and constraints, and before the student starts building. It is mentor-facing — students see only its downstream outputs (the PRD, the writing scaffolds), not the ideation transcript.

2. Architecture



The toolkit is a layered prompt system, not application code. Evidence (reviewer PDFs, accepted-paper metadata, Notion project outcomes) is read at design time — I mine it through a parallel corpus-analysis Claude chat and bake the patterns into the system prompt as one-line guardrails.

The system prompt is then pasted into a per-student Claude Project. The Project pulls dynamic state from Notion via a [SYNC] command rather than pre-loading project state into every conversation. Three classes of output come out: ideation artifacts (seed portfolio, feasibility scorecard, 8-week plan, PRD), draft-audit artifacts (per-venue writing feedback), and submission artifacts (the nhsjs-tools Python package, the only piece of this with actual Python in it).

3. Instructions — the v6 system prompt

The full v6 prompt is ~29 KB and lives at IdeationBOT-v6.md in this repo. Six excerpts below show the parts that do most of the work, with the reviewer-corpus evidence each one is calibrated against.

3.1 Structural saturation, not topical saturation, is the primary rejection signal

1. Structural saturation check. Before locking a seed, name the form. If the form is "model A vs. model B on dataset C, accuracy reported" with no further move, the seed is structurally saturated regardless of topic. Reject or reframe. Layering (b), (c), (f), or (g) on top of a model comparison rescues it.

Saturated topical clusters in the NHSJS 2025–2026 corpus:
stock/market ML (10 papers), sentiment analysis (5),
oncology-ML reviews (5). Exoplanet ML is NOT saturated (1 paper)
– the saturation pattern is structural, not topical.

Why. The corpus analysis (n=13 reviewer PDFs, 431 classified objections) showed reviewers rejecting on form, not topic. A topic that looks “overdone” can still publish if the form is interesting; a fresh topic in a saturated form will not. Hardcoding the saturated topical clusters by absolute paper-count makes the screen evidence-attested rather than gut-feel.

3.2 Creative-move taxonomy (a)-(h), with (e) auto-rejected

- (e) None / generic. Model X on dataset Y with no further move.
Example: bare "try a different classifier on Kepler exoplanet data." REJECT for NHSJS.
- (f) Operationalize a vague construct. Define a number for an informally-discussed concept using public data.
Highest-praised, least-used move at NHSJS. Push toward this when the topic permits.
- (g) Predict-explain framing. Pitch the paper as interpretability – what the model reveals about the phenomenon – rather than prediction. Use as the default voice when accuracy isn't the headline. Pre-commit at design as the Plan B for any predictive seed.

(f) and (g) are the under-used moves NHSJS reviewers consistently praise. Encourage them actively. Don't default to (c) or (d) just because they're easier to generate.

Why. Tagging the move forces the bot to make the novelty claim visible. (e) is the failure mode the corpus shows reviewers rejecting most consistently — “any new claim is being tested at all?” — so the tag also acts as a gate. (f) and (g) are the moves the reviewer corpus praised but the project corpus underused; tagging them surfaces the opportunity each time.

3.3 Predict→explain pivot pre-committed at design time

- Plan B if predictive accuracy is mediocre: One line – "if accuracy falls below threshold X, paper reframes as interpretability study: what the model's [coefficients / feature importances / clusters] reveal about [phenomenon]." Pre-commit the pivot at design. Skip this field if the seed isn't predictive.

Why. Salvage moves discovered at write-up are the visible scar on rejected papers — the framing wobbles, the abstract overclaims, the results section reads like it was reframed in a panic. Pre-committing the pivot at seed stage means a mediocre accuracy number is a planned outcome, not a crisis. This is the move that took a student's $R^2 = 0.143$ paper from "this didn't work" to "this is what the model tells us about IPL bowling," and it published.

3.4 Cross-domain integrity check (for (d) seeds)

Cross-domain transfer integrity (for (d) seeds only). Name (1) the new measurable claim the transfer makes in the target domain, and (2) confirm that every transferred parameter is grounded in target-domain data, not chosen for convenience.

Pure analytic equivalence (transferring a model that's mathematically the same as the target domain's existing tool) fails NHSJS – reviewers call it "trivial." Keeping source-domain branding while breaking source-domain math (e.g., "Sgr A-mass black hole" with a fitted gravitational parameter) also fails – reviewers demand either consistent dimensional relationships or explicit relabeling as a nondimensional toy.

Why. Cross-domain transfer ((d) seeds) is the highest-upside creative move and the highest-variance. The corpus surfaced two specific failure modes: pure analytic equivalence (the project is the source-domain model with target-domain labels) and broken-math-with-retained-branding (calling something a black hole when the gravitational parameter has been fitted to data). Both are reviewer-attested. The guardrail forces a choice: preserve the math, or relabel as a toy. No middle ground.

3.5 IB-vs-Writing-Prompt split rule

The rigour set up during ideation (primary outcome, stats plan, scope boundary, a-priori limitation, reproducibility, reference-bucket projection) lives in the ideation conversation and in Outputs B, E, and F – it does not need to appear as new sections in the PRD. Where these ideas fit naturally inside an existing PRD section, write them in. Do not force them if they don't fit.

Why. v5.2's failure was that I tried to encode every reviewer-attested rigor item as its own PRD section. The PRD doubled in length; mentors stopped reading it; students wrote against a schema they didn't understand. v6 reverts: design-time logic stays in IdeationBOT and surfaces through the ideation conversation; the PRD inherits only the bits that fit its existing structure. Manuscript-hygiene logic (citation style, prose rules, bullet bans) lives in the per-venue writing prompts. Anything that affects whether a project is *worth doing* lives in IdeationBOT; anything that affects whether a *finished* project is *worth submitting* lives in the writing prompts. Two surfaces, no overlap.

3.6 Reviewer-corpus-attested guardrails

2. Don't overclaim. Every seed must be constructable so the final

claim is exactly what the data support. Causal verbs ("X causes Y," "X is the reason Y," "X is the best approach") require interventional designs with randomization – otherwise rephrase to associational, correlational, or predictive.

Reviewers flag causal overreach in 45% of NHSJS papers and 41% of JHSS papers; the highest-volume NHSJS-reviewer phrase is "Do not overclaim. Just report what you found."

Why. Every percentage in v6's guardrails is grounded in the classified objection corpus. "Don't overclaim" isn't asserted as a maxim — it's asserted as the literal modal objection across 431 reviewer comments. The rule of thumb I held to in v6: if a guardrail couldn't be paired with a percentage from the corpus, it didn't earn its place in the prompt.

4. Context the tool has access to

The system prompt itself does not retrieve from the corpora at runtime. Retrieval happens upstream, in a separate Claude conversation that holds the corpora as native context, where I run a fixed set of structured queries against them before updating the system prompt. The corpora are the design-time evidence; the system prompt is the compiled output.

NHSJS reviewer-decision PDFs. 13 PDFs, one per Athena-mentored manuscript submitted across 2025–2026 and reviewed by NHSJS. Each follows the same template (Summary → Major Issues → Minor Issues, numbered). I extracted them with PyMuPDF, regex-split into objection items, and classified each objection against 60 priority-ordered subcategory rules mapped to 12 top-level categories (METHODOLOGY, STATS_RIGOR, REPRODUCIBILITY, PRESENTATION, NOVELTY, OVERCLAIM, SAMPLE_SIZE, REASONING, FORMATTING, LITERATURE, SCOPE_CREEP, OTHER). The output is `nhsjs_reviewer_objections.csv` (431 rows, one objection per row) plus the `nhsjs_peer_review_patterns.md` report. The CSV is what every percentage in the v6 prompt is grounded in. Mean ≈ 33 objections per paper; range 16–73; no first-submission accepts in the corpus.

NHSJS accepted-paper records. 685 papers from the NHSJS 2025–2026 published archive, each row carrying title, link, submission date, acceptance date, review days, classified subject (primary + secondary, e.g., "ML/CS / Medicine-Health"), and topic keywords. This is the corpus the structural-saturation check is calibrated against: I ran category aggregation over it to find which forms NHSJS publishes a lot of (stock/market ML, sentiment analysis, oncology-ML reviews) versus which forms are still open (exoplanet ML at $n=1$; many physics-instrumentation niches at $n=0$). The classification CSV (`nhsjs_classified.csv`) is queried at design time to update the v6 prompt's saturated-cluster line whenever the corpus is refreshed.

Pangea project records (Notion). 17 active student projects tracked in a structured Notion database with fields including starting interest, locked research question, novel twist, dataset, methodology, creative-move tag (a–h), outcome (published / under review / stalled), and an "honest read" field where I record what actually distinguished the projects that published from the ones that stalled. The toolkit's runtime per-student Claude Project pulls a student's record on demand via a [SYNC] command (not pre-loaded into context). The aggregate distribution — 5 published, 4 stalled, rest active — is the design-time evidence behind the data-operationalization gate and the reference-bucket viability check: stalled projects in this corpus failed those two checks more often than any other single factor.

5. Iteration discipline

The lineage is v3 → v4 → v5 → v5.1 → v5.2 → v6, with every prior version preserved verbatim in `_archive/`. v3 was the first version that treated NHSJS as primary venue and added IJHSR-secondary routing. v4 introduced the seed-portfolio voice (each seed as a strategic editorial recommendation, not a schema-filled table) — this is the voice the toolkit's mentor-facing outputs still use. v5 added cross-venue routing logic and the first pass at JHSS reviewer-pattern mining. v5.1 tightened the v5 logic. v5.2 was supposed to be the destination: it added a heavier PRD schema, encoded every reviewer-corpus insight as a dedicated PRD section, and standardized the rigor-checks as fields.

I killed v5.2 after running it for two weeks. The problem was not that v5.2 was wrong — every section it added was traceable to a real reviewer objection. The problem was that v5.2 crowded out v4's editorial voice. Mentors stopped reading the PRD because it had grown to twice its length. Students wrote against a schema they didn't understand. The reviewer-attested checks were nominally present, but nothing was happening with them because the design-time conversation no longer had room for the strategic-editorial back-and-forth that gets a mentor to actually argue for a (g) framing over an (e) baseline. v6 reverts to v4's voice and adds only the guardrails that earned their place by being attached to a specific reviewer-corpus percentage. Six guardrails survived this cut; another nine were dropped. The v5.2 kill is the cleanest single piece of evidence-led design discipline in the lineage — and the file is in `_archive/IdeationBOT v5.2.md` if anyone wants to read what got cut and why.

6. Sample input and output

A real session, lightly edited. The student arrived with a too-saturated framing; the toolkit produced a seed portfolio, named a top pick that applied the predict→explain pivot, and the project subsequently published in IJHSR.

Input (mentor message at top of fresh ideation chat)

[paste IdeationBOT v6.md as system prompt]

Mentor message:

Grade 11 student. Strong math (school olympiad), competent Python, no GPU. Read about the Kepler mission and wants to do something with exoplanet detection using neural networks. Has heard "convolutional neural network" and wants to use one. Has 8 weeks. NHSJS target.

This is Mode B – vague interest, needs a seed portfolio.

Output (seed portfolio, abbreviated)

Assumptions: NHSJS primary; T1 laptop, no GPU; 8 weeks; novice-to-intermediate ML. The literal framing – "detect exoplanets with a CNN" – is structurally saturated regardless of topic (model A vs B, accuracy reported). Exoplanet ML itself is NOT saturated at NHSJS (only 1 paper in the 2025–2026 accepted corpus). So we rescue the topic by layering a creative move; we do not run the bare CNN.

Seed portfolio (4 seeds, all rescuing the topic):

SEED 1 – What features of a Kepler light curve carry the most predictive signal, and what does that tell us about transit geometry?

Anchor papers: Shallue & Vanderburg 2018 (CNN baseline);
Pearson 2019 (lightweight CNN on Kepler quarters);
Yu 2019 (interpretability of transit-signal features).
All open access. (3 anchors – solid niche.)

Headline: "A simple classifier trained on Kepler light curves reveals which features of the curve – depth, duration, asymmetry – carry the most predictive signal, and what those features imply about the transit geometry."

Creative move: (g) Predict-explain. The paper is interpretability, not accuracy. NHSJS publishes (g) framings consistently.

Core question: Among engineered transit features, which carry the most predictive weight for a confirmed-planet vs. false-positive classification, and what does the ranking imply about the underlying transit geometry?

Primary outcome & stats plan: AUC-ROC on a held-out test split, plus permutation-feature-importance ranking with bootstrap CIs. Primary claim is the ranking, not the AUC.

Plan B if accuracy mediocre: Already the framing – the AUC is diagnostic, not the headline. No salvage needed.

Scope boundary: This project claims feature-importance ranking on one Kepler quarter; it does not claim a production detector or generalization across mission phases.

Data/compute plan: Pre-confirmed Kepler quarter (NASA Exoplanet Archive), ~10K labeled curves. scikit-learn random forest + logistic-regression baseline on engineered features. T1 laptop sufficient. No CNN – the headline doesn't require one.

A-priori limitation: Engineered features lose information vs. raw light-curve representations; the ranking is conditional on the feature set chosen.

Feasibility note: Risk = feature engineering takes longer than estimated. Fallback = restrict to depth + duration + ingress asymmetry only (literature precedent).

Reference bucket check: Methods/tools 4, Statistics 3, Foundational 3, Contemporary 4, Competing 3, Evaluation 3 = 20 refs. Clears NHSJS's 20+ primary-peer-reviewed bar.

SEED 2 – (b) Reframe: Train on one Kepler quarter, test on another – how does temporal coverage affect generalization?
[Same structure as Seed 1, abbreviated for this excerpt]

SEED 3 – (h) Feasibility: A laptop-only Kepler classifier – what

accuracy is achievable without a GPU, and what's the opportunity cost in feature space?

SEED 4 – (f) Operationalize: Define "shallow transit" quantitatively from depth + duration; what fraction of NASA Exoplanet Archive confirmed planets meet the definition?

Top-3 recommendation:

1. SEED 1 – (g) predict-explain on Kepler. Strongest move for NHSJS; topic clear of saturation; primary outcome is the ranking, not the AUC, so accuracy doesn't kill the paper. This is the one I'd back.
2. SEED 4 – (f) operationalize "shallow transit." Also strong; less ML, more measurement; reference bucket harder to hit.
3. SEED 3 – (h) feasibility as contribution. Safe; weaker for an Olympiad-strong student who can carry Seed 1.

Competitive-landscape note: 1 NHSJS-accepted exoplanet ML paper in 2025–2026; the topic is open. JHSS prior in last 24 months: also sparse.

Which of these feels right? Say a number or tell me what to change.

The student picked Seed 1. Eight weeks later the paper was at draft. It published in IJHSR shortly after.

7. nhsjs-tools — the submission-formatter component

The only part of this toolkit with actual Python in it. github.com/adhvaryu-athena/nhsjs-tools — published to PyPI as nhsjs-tools and deployed as a Streamlit web app.

What it does.

Three submission paths, each end-to-end:

- **LaTeX → Online format.** Overleaf .zip or .tex → NHSJS Online .docx with ((full citation)) brackets, embedded figures, auto-numbered captions, resolved cross-references.
- **Standard → Online format.** NHSJS Standard .docx (real OOXML superscript citations + References section) → Online .docx with citations expanded.
- **Revision → All Outputs.** A revision.md (sequence of anchored patches) + original Standard .docx → all 5 NHSJS submission files (Standard-Tracked with real Word tracked-changes, Standard-Clean, Online, Response-Letter, Student-Handoff) + a 13-check self-audit report.

The v2.2 patch model is the load-bearing anti-hallucination move: revision.md is a sequence of anchored patches (FIND_REPLACE, INSERT_AFTER_PARAGRAPH, REF_LIST with ADD / REPLACE_AT / RENUMBER, etc.) against the live original docx, not a re-emission of the manuscript. The builder will not match a FIND that doesn't appear verbatim in the source — so the bot can never silently rewrite untouched text. The regression gate is a reconstruction of a real mangroves paper (73 patches, 21k chars revised) which reconciles at 1.000 ratio against the human-produced final.

Install.


```
pip install nhsjs-tools
```

Streamlit web app:

```
streamlit run streamlit_app.py
```

One-command Claude-chat bootstrap:

```
!pip install nhsjs-tools -q  
!nhsjs-start
```

Where it sits in the toolkit’s workflow. After ideation produces the PRD, after the writing-prompt scaffolds produce a draft, after NHSJS returns reviewer comments and the Revise-After-Review v2.2 prompt produces the patched `revision.md` — `nhsjs-tools` turns that markdown into the actual files NHSJS requires for submission. It is the only part of the toolkit a student or mentor runs locally rather than inside Claude; everything upstream of it lives in prompts.

It cut average per-paper formatting time from 2–3 hours of hand-formatting to ~30 minutes of patch authoring + tool run, and the 13-check self-audit catches the formatting failures (orphan refs, citation-numbering drift, vanishing content under Accept-All, “my mentor” leaks in anonymous submissions) that previously surfaced only when the editor flagged them.

Scope and authorship

Solo work. I designed the lineage, ran the corpus analysis, wrote every version of the system prompt, and built `nhsjs-tools`. The reviewer-corpus PDFs are Athena-mentored student submissions; the accepted-paper corpus is public NHSJS archive content; the Pangea Notion database is internal Athena. Nothing here is confidential — system prompts, corpus details, and project case names are all included as-is. The toolkit is internal infrastructure at Athena; this repository is the canonical reference for what it does and how it works.